

Parallelising loops

Unit 2.5

Parallelising loops

- In order to do useful work in parallel we need a way to distribute the workload between threads
- One way to do this in OpenMP is to parallelise loops
- Different threads carry out different iterations of the loop
- We can parallelise a `for` loop from inside a parallel region:

```
#pragma omp for
```

- We can start a parallel region and parallelise a `for` loop:

```
#pragma omp parallel for
```

Parallel loop example: calculating $y = \ln(x)$

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int main(){

    const int N=12;
    float x[N],y[N];

#pragma omp parallel ← Start parallel region
{
#pragma omp for ← Parallelise the for loop
    for (int i=0; i<N; i++){
        printf("Thread %d calculating log(%d)\n", omp_get_thread_num(), i+1);
        x[i]= (float) (i+1);
        y[i]= log(x[i]); ← log is natural log (ln) in C
    }
}

    return 0;
}
```

We can combine the two directives to start a parallel region and parallelise the `for` loop

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int main(){

    const int N=12;
    float x[N],y[N];

    #pragma omp parallel for
    for (int i=0; i<N; i++){
        printf("Thread %d calculating log(%d)\n", omp_get_thread_num(), i+1);
        x[i]= (float) (i+1);
        y[i]= log(x[i]);
    }

    return 0;
}
```



Order of loop iterations

- When the loop is parallelised the iterations will not take place in the order specified by the loop iterator
- We can't rely on the loop iterations taking place in any particular order
- The results are stored in arrays which **do** hold the results in the order we require
- If we want to print out the results of our calculation in order we will need a second sequential (not parallelised) loop

```
#include <stdio.h>
#include <math.h>
#include <omp.h>
```

```
int main(){
```

```
    const int N=12;
    float x[N],y[N];
```

```
#pragma omp parallel for
    for (int i=0; i<N; i++){
        x[i]= (float) (i+1);
        y[i]= log(x[i]);
    }
```

← This loop is parallel

```
    for (int i=0; i<N; i++){
        printf("%g %g\n", x[i], y[i]);
    }
```

← This loop is sequential

```
    return 0;
}
```

Unit summary

- We can parallelise a `for` loop in a parallel region with the directive: `#pragma omp for`
- We can start a parallel region **and** parallelise a `for` loop with the directive: `#pragma omp parallel for`
- In a parallel loop different threads carry out different loop iterations
- Parallel loop iterations are not carried out in the order specified by the loop iterator
- A sequential loop is required to write out results in order